Sprint 8 · Reto «Duelo de modelos»

Ficha completa del reto, para consultar. Para trabajar con ella abierta, usad el reto en una página.

- Volver al README del sprint · README del sprint en PDF · ecofriendly
- **Esta ficha en PDF:** a color · ecofriendly, para imprimir
- Si es vuestro primer reto: Cómo se trabaja un reto.

RETO Ficha del reto

1 Qué os pide el cliente

Qué os pide el cliente. TurisData ya sabe que se pueden predecir las cancelaciones de reservas (lo visteis en el Sprint 7). Ahora el gerente pregunta: **¿qué modelo ponemos en producción?** Duda entre tres cosas: «*¿el más preciso? ¿el que yo pueda entender? ¿el más barato de mantener?*». Vuestro equipo tiene que **enfrentar varios modelos** (árboles, bosques, *boosting* y SVM), medirlos todos con las mismas reglas en una tabla de resultados (el *leaderboard*) y **elegir un campeón con argumentos**. La idea clave: en la vida real **el modelo más preciso no siempre es el mejor**; uno un poco peor, pero que se entiende y se mantiene fácilmente, puede ser la mejor decisión. Por eso se decide con **tres criterios**: rendimiento, explicabilidad y coste.

Cómo se trabaja. No hace falta experiencia previa: el cuaderno `S08_06` ya trae las herramientas para medir todos los modelos igual, y vosotros rellenáis los huecos paso a paso. Hay una regla de oro: **el test se usa una sola vez, solo con el campeón**; probar varios modelos en el test y quedarse con el mejor es hacer trampa. El trabajo completo son unos **150 minutos de equipo** (cinco pasos): en clase tenéis 10 minutos de planning (día 1), **75 minutos de reto + 35 de entrega, review y retro** (día 2); lo que no quepa va a casa o a la 6.ª hora. El equipo decide hasta dónde llega: mínimo, estándar o extra.

2 · Recursos del reto

Recurso	Para qué lo usaréis	Dónde está
Google Colab (el navegador, sin instalar nada) con las librerías `scikit-learn`, `xgboost` y `lightgbm`	Ejecutar el cuaderno del reto	Ya instaladas en Colab
El cuaderno del reto `S08_06_leaderboard_plantilla`	Medir todos los modelos igual, guardar `leaderboard_S08.csv`, calcular la puntuación global y evaluar el campeón en el test	Colab. Su código explicado
Cuadernos de modelos: `S08_01` árboles, `S08_02` bosques, `S08_03` *boosting* y `S08_04` SVM	Las pistas para cada familia de modelos	Colab; su código explicado
Cuaderno de búsqueda de hiperparámetros `S08_05`	La guía para el paso 3 (`GridSearchCV` y `RandomizedSearchCV`). Es el cuaderno **del reto** que se hace **en casa la tarde del 11/11** (≈ 40 min; para el paso 3 os bastan las secciones 2 y 3): venid con él hecho al paso 3	Colab; su código explicado
Presentaciones y apuntes	Repasar cómo funciona cada modelo: árboles, ensembles, SVM y apuntes	Carpeta `1_Teoria`

Recurso	Para qué lo usaréis	Dónde está
Plantilla de la *model card* `model_card_plantilla.md`	La «ficha técnica» del campeón, con sus apartados	En GitHub: *Raw* y copiar el texto
Plantilla del *leaderboard* `leaderboard_plantilla.csv`	Ver qué columnas tiene la tabla (el cuaderno ya crea la vuestra)	Solo para consultar
Los datos `reservas_turisdata.csv`	El problema: predecir la columna `cancelada`. Son reservas **sintéticas**	El cuaderno los descarga solo si no los encuentra
Cómo entregar en GitHub	Subir los ficheros a la carpeta `sprint-08/`	Guía de entrega
Autoevaluación y coevaluación	Cada persona rellena su hoja	Hoja del sprint
Un asistente de IA (opcional)	Entender modelos y errores, y preparar la defensa	Con las reglas de la «Ayuda 1» del paso a paso

3 · Qué entregar

Este apartado responde a **dos preguntas distintas**, una tabla para cada una:

Tabla A · Los ficheros: qué ficheros hay, de dónde sale cada uno (ya hecho, lo modificáis, lo creáis o lo genera el cuaderno) y **cuáles se suben y adónde**.

Tabla B · Los requisitos: qué tiene que cumplir lo que entregáis y hasta dónde llegar (mínimo, estándar, extra). **Es lo que se evalúa.**

En total, al final del sprint se suben a `sprint-08/` : 4 ficheros del equipo (`leaderboard_S08.csv` , `model_card.md` , `reto_S08.ipynb` , `README.md`) **+ 1 autoevaluación y coevaluación por persona**, es decir, **4 + tantos ficheros como personas haya en el equipo** (un equipo de 3 sube 7). «El reto» no es un fichero aparte: son esos cuatro ficheros. Los datos no se suben: el cuaderno los descarga solo. La *review* y la *retro* se hacen de palabra: no se suben.

Tabla A · Los ficheros: de dónde sale cada uno y cuáles se suben

Todo lo que se sube va al repositorio privado del equipo (`ifcd107-equipo-…`), en la carpeta **`sprint-08/`** .

Fichero	Origen	Qué hacéis vosotros	¿Se sube?
`leaderboard_S08.csv`	**Lo genera** el cuaderno: cada `registrar(...)` añade o actualiza una fila	Registrar vuestros modelos en el cuaderno y, al acabar, descargar el CSV desde el panel de archivos de Colab. No se escribe a mano	**Sí** · `sprint-08/`
`reto_S08.ipynb`	**Lo modificáis.** Parte de `S08_06_leaderboard_plantilla` : las herramientas ya vienen hechas	Rellenar los huecos `TODO` , añadir las búsquedas de hiperparámetros y dejarlo ejecutado de arriba abajo	**Sí** · `sprint-08/`
`model_card.md`	**Lo creáis a partir de una plantilla.** Os damos `model_card_plantilla.md` con apartados	Copiarla como `model_card.md` y rellenarla con **vuestros** números	**Sí** · `sprint-08/`
`README.md`	**Lo creáis.** No hay plantilla: el paso 5 os da el guion	Escribirlo: qué se entrega, quién hizo qué, cómo se reproduce, uso de IA y revisión con la rúbrica	**Sí** · `sprint-08/`

Fichero	Origen	Qué hacéis vosotros	¿Se sube?
Autoevaluación y coevaluación (`autoevaluacion_nombre.md`)	**Ya hecha, en blanco**	Cada persona copia la plantilla con su nombre y la rellena; la coevaluación va **dentro del mismo fichero**	**Sí**, **una por persona** (no una por equipo) · `sprint-08/`
`leaderboard_plantilla.csv`	**Ya hecho.** Solo enseña las columnas de la tabla	Nada: el cuaderno crea el vuestro	**No**
`reservas_turisdata.csv`	**Ya hecho.** Los datos (sintéticos)	Nada: el cuaderno los descarga y no se cambian	**No**, ya están en el repositorio del curso

Tabla B · Los requisitos: qué tiene que cumplir y hasta dónde llegar

Requisito	Qué se pide exactamente	Nivel	Dónde se comprueba
Pesos de los criterios	Pesos de rendimiento, explicabilidad y coste que sumen 100 %, acordados con la «voz del cliente» y **escritos con su justificación**	Mínimo	`model_card.md` (apartado 3) y `reto_S08.ipynb`
Leaderboard	Al menos **6 modelos** medidos igual (referencia o *baseline*, logística, árbol, bosque, *boosting* y SVM), de al menos 4 familias y **máximo 8**. Mismos datos, folds y métrica: no se toca la celda de herramientas	Mínimo	`leaderboard_S08.csv` y `reto_S08.ipynb`
Campeón elegido	El campeón sale de `puntuacion_global` con vuestros pesos y se explica con una frase que nombra los tres criterios. La IA no decide	Mínimo	`model_card.md` (apartado 3)
Test una sola vez	El AUC de test se calcula **solo para el campeón y una sola vez**: la columna `auc_test` tiene valor en una única fila	Mínimo	`leaderboard_S08.csv` y `reto_S08.ipynb`
Cuaderno ejecutable	`reto_S08.ipynb` se ejecuta entero con *Reiniciar y ejecutar todo*, sin errores, con `SEMILLA = 42`	Mínimo	`reto_S08.ipynb`
***Model card* (versión corta)**	Resumen, campeón, por qué este y no otro, AUC de test y usos previstos y **no** previstos	Mínimo	`model_card.md`
README corto	Qué se entrega, quién hizo qué y cómo abrir y ejecutar el cuaderno	Mínimo	`README.md`
Entrega en GitHub	Los cuatro ficheros dentro de `sprint-08/`, subidos desde las cuentas de personas distintas, con mensaje claro, sin claves ni datos personales	Mínimo	Historial del repositorio del equipo
Búsqueda de hiperparámetros	En **dos familias** de modelos, con `GridSearchCV` o `RandomizedSearchCV`, validación cruzada y **solo sobre train**. Cada búsqueda, 5 minutos como máximo en Colab. Los mejores modelos, en el leaderboard	Estándar	`reto_S08.ipynb`
Análisis de sensibilidad	Probar otros conjuntos de pesos (el cuaderno trae tres de ejemplo) y contar si el campeón cambia	Estándar	`reto_S08.ipynb` y `model_card.md` (apartado 3)
***Model card* completa**	Los diez apartados de la plantilla con **vuestros** datos: modelo, por qué, datos y límites, métricas con incertidumbre y test, usos previstos y no previstos, explicabilidad, riesgos (país de origen) y mantenimiento (deriva)	Estándar	`model_card.md`

Requisito	Qué se pide exactamente	Nivel	Dónde se comprueba
README completo	Versiones de las librerías, `SEMILLA = 42`, orden de ejecución, y uso de IA (qué pedisteis y qué revisasteis)	Estándar	`README.md`
Revisión y autoevaluación	Alguien que **no escribió esa parte** revisa el cuaderno, la *model card* y el README con la rúbrica (justo debajo de estas tablas) y anota nivel y prueba en el README; cada persona sube su hoja de autoevaluación y coevaluación	Estándar	`README.md` (sección «Revisión con la rúbrica») y `sprint-08/`
Extras	Importancia de variables del campeón explicada al cliente · comparación con incertidumbre (varias semillas) · plan de mantenimiento del modelo	Extra	`reto_S08.ipynb` · `model_card.md`

Rúbrica del reto · cómo se puntúa y cómo se revisa

La usan **dos veces**: el equipo, para revisar su propio trabajo antes de entregar (es lo que dice la *Definition of Done*), y el docente, para puntuar. Se mira **cada criterio por separado** y se pone el nivel de la columna cuyo texto **se cumple entero**: si lo de un nivel se cumple a medias, es el nivel de abajo. **0** significa que no se ha entregado nada de ese criterio.

Orientación: lo que pide el nivel mínimo sale en un **2**, el estándar en un **3** y los extras en un **4**.

Criterio	1 · Empezado	2 · Mínimo	3 · Estándar	4 · Extra
Corrección técnica (¿funciona?)	Hay cuaderno, pero el leaderboard tiene menos de 6 modelos, se ha cambiado la celda de herramientas o no se ejecuta	6 modelos medidos igual; campeón elegido con `puntuacion_global`; AUC de test calculado una sola vez y solo para el campeón	Además, búsqueda con validación cruzada solo sobre train en dos familias, sensibilidad de los pesos probada, 4 familias o más y cuaderno que se ejecuta entero	Además, comparación con varias semillas o importancia de variables del campeón que cuadra con los resultados
Reproducibilidad (¿lo puede repetir otra persona?)	Solo funciona en la sesión de Colab de quien lo hizo, o no hay README	El README dice qué se entrega y cómo abrir el cuaderno, pero faltan versiones, semilla u orden de ejecución	Otra persona, siguiendo solo el README, ejecuta *Reiniciar y ejecutar todo* y obtiene **el mismo leaderboard y el mismo campeón**; los ficheros están en `sprint-08/`, subidos por personas distintas y sin claves	Además, el README da las versiones exactas de las librerías, cuánto tarda en Colab y qué ficheros salen
Análisis y comunicación al cliente (¿lo entiende quien no programa?)	La *model card* está vacía o copiada de la plantilla; solo hay cifras	Dice cuál es el campeón y su AUC de test, pero no explica por qué ganó	Explica por qué ganó y **por qué no ganó otro** con los tres criterios y los pesos justificados, qué pasa si cambian los pesos y los usos previstos y no previstos, sin jerga	Además, explica al gerente en palabras qué variables pesan más, o da una recomendación con sus límites («si el cliente valorara solo el rendimiento, elegiríamos…»)

Criterio	1 · Empezado	2 · Mínimo	3 · Estándar	4 · Extra
IA responsable (¿se sabe qué hizo la IA y qué riesgos tiene el modelo?)	No hay apartado de riesgos ni se dice si se ha usado IA	Menciones genéricas («puede haber sesgos») y dice que se ha usado IA, sin detalle	Riesgos concretos de este modelo (el país de origen como variable sensible, la deriva y qué se medirá, quién revisa las decisiones); dice **qué se pidió a la IA, qué se revisó y qué se cambió**; los datos son sintéticos	Además, mide algo (por ejemplo, el AUC por país o canal) o fija un plan de mantenimiento con un umbral para reentrenar y una persona responsable

Cómo se revisa (unos 10 minutos, antes de subir; lo hace **una persona que no escribió esa parte**):

1. **Ejecutar** `reto_S08.ipynb` en Colab con *Entorno de ejecución → Reiniciar y ejecutar todo*, siguiendo solo lo que dice el README. Anotar si algo falla y si sale el mismo leaderboard y el mismo campeón que dice la *model card*. Frase de prueba: «2 · se ejecuta entero, pero el README no dice el orden».
2. **Contar** en `leaderboard_S08.csv`: modelos (6 a 8), familias (4 o más) y filas con `auc_test` (solo una). Buscar en el cuaderno que `X_test` no aparece en las búsquedas. Frase de prueba: «3 · 7 modelos, 5 familias y `auc_test` en una sola fila».
3. **Leer la *model card* como si fuerais el gerente**: ¿se entiende por qué ganó y por qué no ganó otro? ¿Dice para qué no usarlo? Subrayar lo que no se entienda. Frase de prueba: «2 · dice el campeón, pero no por qué no ganó el *boosting*».
4. **Poner un nivel (1 a 4) en cada criterio, con una frase de prueba** (por ejemplo: «3 · busca en dos familias, pero la sensibilidad solo prueba un conjunto de pesos»). Sin prueba escrita, el nivel no vale. Para IA responsable: «¿dice qué se pidió, qué se revisó y qué riesgo concreto tiene este modelo?».
5. **Apuntar el resultado** en la sección «Revisión con la rúbrica» del `README.md` y corregir lo más fácil de arreglar. Lo demás se anota como mejora.

Cada persona, además, se puntúa a sí misma en su hoja de autoevaluación y coevaluación con **esta misma rúbrica**.

---

4 · Guía paso a paso

Son cinco pasos, de **150 minutos en total**. Cada uno dice **qué hacéis** (la lista corta), **cómo se hace** con el detalle, **qué deberíais ver** y **cuándo está hecho**. Las ayudas (IA, errores, atascos) están al final. El calendario da menos tiempo de reto del que suman los pasos: el día 1 solo hay 10 minutos (el planning, donde se acuerdan los pesos; los otros 10 son para escribirlos justificados al empezar el día 2) y el día 2 hay 75 minutos de reto, más la entrega, la review y la retro. Los 65 minutos que faltan se hacen al final del día 2, en la **consolidación** (13:00–13:30) y en la **6.ª hora** (hasta las 14:30); la subida puede hacerse hasta esa hora. Si el tiempo aprieta, completad el nivel mínimo y dejad lo demás para casa.

Paso 1 · Planning y pesos de los criterios (día 1, 20 minutos)

1. Repartir roles y tareas, y apuntarlas en el tablero.
2. Leer esta ficha y decidir hasta qué nivel queréis llegar (, o).
3. Acordar los **pesos** de los tres criterios con la «voz del cliente» (el docente) y justificarlos.
4. Escribir los pesos y su justificación en un sitio común: los usaréis en los pasos 4 y 5.

Cómo se hace

- **Los tres criterios.** Cada modelo se puntúa en tres cosas, y vosotros decidís cuánto vale cada una (los pesos suman 100 %):

Criterio	Qué mide	Peso (ejemplo)
Rendimiento	El AUC en validación cruzada, con su desviación (media ± desviación)	50 %

Criterio	Qué mide	Peso (ejemplo)
Explicabilidad	¿Puede el gerente entender **por qué** el modelo avisa? Se puntúa de 1 (caja negra) a 5 (se lee directamente)	30 %
Coste	Tiempo de entrenamiento y de predicción, tamaño del modelo y esfuerzo de mantenimiento	20 %

- **Justificarlos.** Los pesos son vuestros; lo importante es que tengan una razón. Preguntad al docente (que hace de cliente): «¿Necesita el gerente explicar a un cliente por qué se le avisa?», «¿hay poco presupuesto para mantener el modelo?». Ejemplo de frase: «Damos 30 % a la explicabilidad porque el gerente necesita entender por qué se avisa».
- **Repartir el trabajo.** Un reparto posible (con 3 personas, la persona D no existe y su tarea se reparte):

Persona	Tarea	Paso
A	Modelos de referencia, lineal, árbol y bosque en el cuaderno	2
B	*Boosting*, SVM y primera búsqueda de hiperparámetros	3
C	Segunda búsqueda de hiperparámetros y análisis de sensibilidad	3 y 4
D (si hay 4)	*Model card* y README	5

Todos revisan el trabajo de otra persona. Para el cuaderno, usad **una sola copia compartida** (Compartir → editor) o una pantalla compartida, y no editéis la misma celda a la vez.

Palabras que vais a usar.

Palabra	Qué significa, en sencillo
Leaderboard	La tabla de clasificación de los modelos, con sus resultados
Familia de modelos	Un tipo de modelo: árbol, bosque, *boosting*, SVM, lineal…
Hiperparámetro	Un ajuste que hacéis vosotros **antes** de entrenar (la profundidad del árbol, el número de árboles…)
Validación cruzada	Evaluar varias veces con particiones distintas del *train* para tener una medida fiable
AUC	Nota de 0,5 (como el azar) a 1 (perfecto): cómo de bien separa las reservas que se cancelan de las que no
Explicabilidad	Poder decir **por qué** el modelo toma una decisión
Model card	La «ficha técnica» del modelo: usos, datos, métricas, límites y riesgos
Deriva	Cuando los clientes cambian su comportamiento con el tiempo y el modelo deja de funcionar bien

Qué deberíais ver. Los pesos escritos (por ejemplo, 50 / 30 / 20) con una frase de por qué, las tareas en el tablero y el nivel objetivo acordado.

Hecho cuando: los pesos suman 100 %, están justificados y cada persona sabe qué le toca.

Paso 2 · Primeros modelos en el leaderboard (día 2, 30 minutos)

1. Abrir `S08_06_leaderboard_plantilla` en Colab, guardar una copia y llamarla `reto_S08.ipynb`.
2. Ejecutar las celdas de preparación y de herramientas **sin cambiarlas**.
3. Registrar el modelo de referencia (*baseline*), la regresión logística, un árbol podado y un bosque.
4. Comprobar que la tabla tiene una fila por modelo.

Cómo se hace

- **Abrir y guardar una copia.** Abrid el cuaderno, pulsad *Archivo* → *Guardar una copia en Drive* y luego *Archivo* → *Cambiar nombre* para llamarla `reto_S08.ipynb`. Trabajad siempre en **vuestra copia**.

- **Ejecutar en orden.** Ejecutad las celdas de arriba abajo; la primera (*Preparación*) siempre antes que las demás. La celda **«Herramientas del leaderboard»** no se toca: es la que garantiza **mismos datos, mismos folds y misma métrica** para todos los modelos.
- **Rellenar los huecos.** Donde hay `# TODO` y `...`, escribid vosotros el código. Cada modelo se registra con el patrón `registrar(evaluar_modelo(nombre, familia, hacer_pipe(...), explicabilidad, notas))`. Las pistas de los cuatro de este paso:

Modelo (nombre)	Familia	Qué le pasáis a `hacer_pipe`	Explicabilidad
`baseline`	`referencia`	`DummyClassifier(strategy="prior")` (siempre predice lo más habitual)	5
`logistica`	`lineal`	`LogisticRegression(max_iter=1000)` con `escalar=True`	4
`arbol_podado`	`arbol`	`DecisionTreeClassifier(min_samples_leaf=80, random_state=SEMILLA)`	5
`random_forest`	`ensemble_bagging`	`RandomForestClassifier(n_estimators=200, min_samples_leaf=5, random_state=SEMILLA, n_jobs=2)`	2

- **Dos tandas.** Los ejercicios 1 y 2 del cuaderno traen una comprobación (`assert`). La del ejercicio 2 pide 6 modelos, así que dará error hasta que estén también *boosting* y SVM (paso 3): es normal. Pistas de cuaderno: `S08_01` (árbol) y `S08_02` (bosque).

Qué deberíais ver. Una tabla ordenada por AUC con cuatro filas. El *baseline* sale con AUC cerca de 0,5 (el cuaderno lo comprueba) y el resto, por encima. Cada fila trae AUC medio, desviación, tiempos, tamaño y explicabilidad. En el panel de archivos de Colab (icono de la carpeta, a la izquierda) aparece `leaderboard_S08.csv`.

Hecho cuando: el leaderboard tiene `baseline`, `logistica`, `arbol_podado` y `random_forest`, y cada persona sabe explicar qué mide cada columna.

Paso 3 · *Boosting*, SVM y búsqueda de hiperparámetros (día 2, 25 minutos)

1. Registrar un modelo de *boosting* y una SVM.
2. Buscar hiperparámetros en **dos familias** (rejilla o aleatoria), solo con train y validación cruzada, sin pasar de 5 minutos cada búsqueda.
3. Registrar los mejores modelos y actualizar el `catalogo` del cuaderno.
4. Comprobar que hay entre 6 y 8 modelos, de al menos 4 familias.

Cómo se hace

- **Los dos modelos que faltan.** Mismo patrón que en el paso 2 (pistas en `S08_03` y `S08_04`):

Modelo (nombre)	Familia	Qué le pasáis a `hacer_pipe`	Explicabilidad
`lightgbm`	`ensemble_boosting`	`lgb.LGBMClassifier(n_estimators=100, learning_rate=0.05, num_leaves=8, random_state=SEMILLA, n_jobs=2, verbose=-1)`	2
`svm_rbf`	`svm`	`SVC(kernel="rbf", C=1, probability=True, random_state=SEMILLA)` con `escalar=True`	1

Podéis añadir también `xgboost` (familia `ensemble_boosting`). La SVM suele ser la más lenta: eso cuenta en el criterio de coste.

- **Elegir las dos familias.** Mirad el leaderboard y buscad en las **dos más prometedoras** (por ejemplo, el bosque y *boosting*, como en `S08_05`). Una persona por búsqueda.

- **Buscar sin hacer trampas.** Usad `GridSearchCV` (prueba todas las combinaciones) o `RandomizedSearchCV` (prueba combinaciones al azar), con `cv=cv`, `scoring="roc_auc"` y `.fit(X_train, y_train)`. **Nunca `X_test`.** Como `hacer_pipe` llama `clf` al paso del modelo, los nombres de la rejilla llevan ese prefijo: `clf__max_depth`, `clf__n_estimators`…
- **Cuidar el tiempo.** Si una búsqueda pasa de 5 minutos, reducid las combinaciones (`n_iter` en la aleatoria, menos valores en la rejilla). Si varias combinaciones quedan casi empatadas, elegid la más sencilla (regla de «1 error estándar» de `S08_05`).
- **Guardar el mejor.** Registrad el modelo con los mejores hiperparámetros con `registrar(evaluar_modelo(...))`. Si usáis el **mismo nombre**, la fila se actualiza; si usáis uno nuevo (`random_forest_ajustado`), se añade (recordad el máximo de 8). Añadid también esos modelos al diccionario `catalogo` (más adelante): lo necesitará el paso 4.

Qué deberíais ver. Entre 6 y 8 filas, de al menos 4 familias (`referencia`, `lineal`, `arbol`, `ensemble_bagging`, `ensemble_boosting`, `svm`). El modelo ajustado suele quedar igual o algo mejor que el de partida, pero no siempre; si no mejora, es un resultado válido y se cuenta.

Hecho cuando: hay entre 6 y 8 modelos de al menos 4 familias, la búsqueda está hecha en dos familias con validación cruzada sobre train y el `catalogo` incluye los modelos ajustados.

Paso 4 · Elegir el campeón (día 2, 45 minutos)

1. Calcular la puntuación global de cada modelo con vuestros pesos y ver quién gana.
2. Probar otros pesos (análisis de sensibilidad) y anotar si el campeón cambia.
3. Evaluar al campeón en el test, **una sola vez**, y guardar el resultado.
4. Escribir tres o cuatro líneas: por qué ganó, por qué no ganó otro y qué cambiaría la elección.

Cómo se hace

- **Puntuación global.** Definid el diccionario `pesos` con las claves `"rendimiento"`, `"explicabilidad"` y `"coste"` (en decimales que sumen 1, por ejemplo 0.5, 0.3 y 0.2) y calculad `lb_pond = puntuacion_global(lb, pesos)`. El campeón es la **primera fila** (`campeon`, el nombre del modelo). Pistas: código explicado del cuaderno.
- **Mirad la incertidumbre.** Un AUC de 0,81 no es mejor que uno de 0,80 si sus desviaciones son parecidas: la diferencia puede ser ruido. Si dos modelos empatan, es un resultado válido: elegid el más simple y explicadlo.
- **Sensibilidad.** Probad al menos otros dos conjuntos de pesos (el cuaderno trae `escenarios`: solo rendimiento, equilibrado y explicabilidad primero) y apuntad en una tabla qué campeón sale con cada uno. Si cambia fácilmente, decidlo: la recomendación depende de lo que valore el cliente.
- **El examen final.** Antes de seguir, comprobad en equipo que el campeón es el definitivo. Luego, en el cuaderno: reconstruid el campeón con `catalogo[campeon]()`, ajustadlo con **todo `X_train`**, calculad `auc_test_campeon` (su AUC sobre `X_test` con `roc_auc_score`) y guardadlo con `actualizar_test(campeon, auc_test_campeon)`. **No se evalúa ningún otro modelo en el test**, y el campeón no se cambia después de ver ese número.
- **Las líneas de la decisión.** Con el ejemplo de este modelo: «Campeón: … (AUC … ± …). No ganó … porque …. Si el cliente valorara solo …, elegiríamos …». Esas líneas van a la *model card*.

Qué deberíais ver. Una línea como `Campeón: … · AUC en validación cruzada 0.… · AUC en test 0.…` (el AUC de test está entre 0,65 y 0,85, y cerca del de validación; si está mucho más abajo, hay sobreajuste). En `leaderboard_S08.csv`, la columna `auc_test` tiene valor en **una sola fila**. Y una tabla de sensibilidad con el campeón de cada escenario.

Hecho cuando: hay un campeón elegido con vuestros pesos, la sensibilidad está probada y apuntada, y `auc_test` aparece en una única fila.

Paso 5 · *Model card*, README y entrega (día 2, 30 minutos)

1. Copiar `model_card_plantilla.md` como `model_card.md` y rellenarla con vuestros datos.
2. Escribir el `README.md`.
3. Pedir a alguien que no escribió esa parte que revise con la rúbrica y anote el resultado en el README.
4. Subir los cuatro ficheros a `sprint-08/`.

Cómo se hace

- **La *model card*.** Descargad la plantilla (en GitHub: *Raw*, copiar, y crear `model_card.md` con *Add file → Create new file*), o editadla en cualquier editor de texto. Debe caber en 1–2 páginas y cada apartado lleva **vuestros** datos (nada de dejar los ejemplos). Para el **nivel mínimo** bastan estos apartados, en pocas líneas:

Apartado de la plantilla	Qué escribís
1. Resumen	Qué predice el modelo, para qué decisión y quién lo usa
2. Modelo campeón	Familia, hiperparámetros finales y versiones de las librerías
3. Por qué este campeón	Vuestros pesos, el resultado del campeón, el mejor rival y las líneas del paso 4
5. Rendimiento	AUC en validación cruzada (± desviación) y **AUC de test (una sola evaluación)**
6. Uso previsto y NO previsto	Para qué sirve y para qué **no** debe usarse (por ejemplo, no denegar reservas ni decidir sin revisión humana)

Para el **nivel estándar**, completad también 4 (datos y límites: son sintéticos), 7 (cómo se explica una predicción a un gerente), 8 (riesgos: el **país de origen** es una variable sensible; qué pasa con una falsa alarma), 9 (mantenimiento: qué mediríais en producción para detectar la **deriva**, cada cuánto se reentrena y quién se encarga) y 10 (cómo reproducirlo).

- **El README.** Tres cosas que no pueden faltar: **qué se entrega y quién hizo qué**; **cómo se reproduce** (abrir `reto_S08.ipynb` en Colab y *Reiniciar y ejecutar todo*; versiones de `scikit-learn`, `xgboost` y `lightgbm`; `SEMILLA = 42`; orden de ejecución); y **uso de IA** (qué pedisteis y qué revisasteis; los datos son sintéticos, sin datos personales). Para saber las versiones, en una celda del cuaderno: `import sklearn, xgboost, lightgbm` y mostrad `sklearn.__version__`, `xgboost.__version__` y `lightgbm.__version__`.
- **La revisión con la rúbrica (5 minutos).** Quien revisa sigue los cinco pasos de «Cómo se revisa» (apartado 3) y añade al final del README esta sección, con un nivel de 1 a 4 por criterio y **una frase de prueba**. Si hay tiempo, arreglad lo más fácil; lo demás queda como «lo primero que mejoraríamos».

```
## Revisión con la rúbrica
**Revisado por:** (nombre) · **Fecha:** (dd/mm/aaaa)

| Criterio | Nivel (1–4) | Prueba: una frase de lo que has comprobado |
|---|---|---|
| Corrección técnica | | |
| Reproducibilidad | | |
| Análisis y comunicación al cliente | | |
| IA responsable | | |

**Lo primero que mejoraríamos:** (una cosa concreta)
```

- **Preparar los ficheros.** En Colab: *Entorno de ejecución → Reiniciar y ejecutar todo* (así queda con los resultados visibles y sabéis que funciona de arriba abajo), después *Archivo → Descargar → Descargar .ipynb*. Descargad `leaderboard_S08.csv` desde el panel de archivos (clic derecho → *Descargar*) **antes de cerrar Colab**: al cerrar la sesión se pierde. Ojo: al reiniciar y ejecutar todo, el cuaderno vuelve a crear el CSV desde cero, así que descargadlo después.
- **Subir los ficheros (la vía del curso, desde el navegador).** Seguid Cómo entregar en GitHub: (1) abrid el repositorio del equipo y **entrad primero en la carpeta `sprint-08`**; (2) *Add file → Upload files*; (3) arrastrad los ficheros; (4) escribid un mensaje corto y pulsad **Commit changes**. Cada subida es un *commit* con el nombre de quien la hace. Un reparto posible: la persona A sube `reto_S08.ipynb` y `leaderboard_S08.csv`, la B sube `model_card.md` y la C sube `README.md`. Respetad los nombres tal como están en la tabla A (el reto fija `leaderboard_S08.csv` y `reto_S08.ipynb`, con la S mayúscula), sin espacios.

Qué deberíais ver. En la carpeta `sprint-08` del repositorio, los cuatro ficheros del equipo y una hoja de autoevaluación por persona (**4 + N**); y en el historial (el icono del reloj), *commits* con nombres distintos.

Hecho cuando: otra persona podría ejecutar el cuaderno siguiendo solo el README y obtendría el mismo campeón, la sección «Revisión con la rúbrica» está rellena y los cuatro ficheros están en `sprint-08/`.

Después, 35 minutos de entrega, review y retro: cada persona rellena y sube su autoevaluación y coevaluación a `sprint-08/`. En la *review* (5 minutos por equipo) enseñáis el leaderboard (la tabla o un gráfico), el campeón y **por qué no ganó otro**, y una frase sobre qué cambiaría la elección si el cliente valorara otra cosa. Preguntas de la retrospectiva: ¿nos sorprendió que un modelo sencillo compitiera con uno complejo? ¿Qué criterio pesó más en nuestra decisión? ¿Qué mediríamos en producción para saber que el modelo se degrada?

Ayuda 1 · Cómo usar la IA en este reto

La IA puede ahorrar tiempo, pero **da respuestas que parecen correctas y no lo son**. Reglas:

- **La IA no decide el campeón:** lo decide el cálculo con vuestros pesos.
- **Todo lo que dé, lo ejecutáis y lo entendéis.** Si el docente pregunta por una línea o por un número de la *model card*, tenéis que saber explicarlo.
- **Declaradlo** en el README: qué pedisteis y qué revisasteis.
- **Nada de datos personales ni claves** en una IA.

Para qué	Ejemplo de petición	Qué comprobáis
Entender un modelo	«Explícame cómo funciona un bosque aleatorio con una analogía de votación de expertos, sin fórmulas»	Que podéis explicárselo al gerente
Entender un hiperparámetro	«¿Qué hace `max_depth` en un árbol? ¿Qué pasa si es muy grande?»	Que probáis valores y veis lo que ocurre (sobreajuste)
Redactar la *model card*	«Ayúdame a redactar la sección de limitaciones. Estos son los datos: *[resumen]*»	Que **todo** lo que escriba coincide con vuestros resultados
Preparar la defensa	«Hazme 5 preguntas difíciles que un gerente podría hacer sobre mi elección de campeón»	Que preparáis la respuesta con **vuestros** datos

Ayuda 2 · Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
Comparar modelos con particiones distintas	Cada persona prueba «por su lado»	Usad el cuaderno plantilla y no cambiéis la celda de herramientas: mismos folds para todos
Ajustar hiperparámetros mirando el test	«Para ver si mejora»	Buscad solo con `X_train` y validación cruzada; `X_test` solo aparece en el paso 4
Evaluar varios modelos en el test y quedarse con el mejor	Curiosidad o prisa	El test se usa **una vez**, con el campeón ya elegido; `auc_test` en una sola fila
Elegir por el AUC más alto sin mirar la desviación	«0,81 es más que 0,80»	Si las desviaciones se solapan, la diferencia puede ser ruido: elegid el más simple
`model_card` genérica	Se copian los ejemplos de la plantilla	Cada apartado con **vuestros** datos y riesgos
Olvidar la explicabilidad y el coste	Solo se mira la precisión	Puntuadlos y ponderadlos con `puntuacion_global`
Búsquedas que duran horas	«Más combinaciones»	Máximo 5 minutos por búsqueda: menos combinaciones o `RandomizedSearchCV` con pocas iteraciones
Se pierde el leaderboard	Colab borra los ficheros al cerrar y `REINICIAR = True` vuelve a empezar de cero	Descargad `leaderboard_S08.csv` antes de cerrar y no editéis el CSV a mano

Error	Por qué ocurre	Cómo evitarlo
El campeón no está en `catalogo`	Se registró un modelo ajustado y no se añadió al diccionario	Añadid cada modelo ajustado a `catalogo` con el mismo nombre del leaderboard

Ayuda 3 · Si os atascáis

1. **No sabéis qué peso poner:** pensad en lo que diría el gerente. Si prioriza entender, sube la explicabilidad.
2. **La búsqueda tarda demasiado:** reducid el número de combinaciones o usad búsqueda aleatoria con pocas iteraciones.
3. **Dos modelos empatan:** es un resultado válido. Elegid el más simple y explicadlo.
4. **Falla una comprobación (`assert`) del cuaderno:** leed el mensaje, que dice qué esperaba, y repasad el ejercicio de esa celda y su código explicado. Preguntad al equipo y, después, al docente.